

COVER SHEET FOR SUBMISSION OF WORK FOR ASSESSMENT



UNIT DETAILS

Unit name **Introduction to Artificial Intelligence** Class Day/time **Monday / 8AM to 10AM**

Unit Code **21st June 2026** **COS30019 A.** Assignment no. **2B** Due date

Name of lecturer/teacher: **Mr. Chee Hui Ang**

Tutor/marker's name

STUDENT(S) DECLARATION AND STATEMENT OF AUTHORSHIP

No.	ID Number	Name	Sign
1	105292899	Aung Moe Thu	
2	104182643	Kaung Htet Nyein	
3	105293041	Show Wai Yan	
4	104992813	Yadanar Theint	

1. I/we have not impersonated or allowed myself/ourselves to be impersonated by any person for the purposes of this assessment.
2. This assessment is my/our original work and no part of it has been copied from any other source except where due acknowledgement is made.
3. No part of this assessment has been written for me/us by any other person except where such collaboration has been authorized by the lecturer/teacher concerned.
4. I/we have not previously submitted this work for this or any other course/unit.
5. I/we give permission for my/our assessment response to be reproduced, communicated, compared and archived for plagiarism detection, benchmarking or educational purposes.
6. I/we understand that:
7. Plagiarism is the presentation of the work, idea or creation of another person as though it is your own. It is a form of cheating and is a very serious academic offence that may lead to exclusion from the University. Plagiarized material can be drawn from, and presented in, written, graphic and visual form, including electronic data and oral presentations. Plagiarism occurs when the origin of the material used is not appropriately cited.

Student signature/s

I/we declare that I/we have read and understood the declaration and statement of authorship.

Table of Content

1. INSTRUCTIONS	3
2. INTRODUCTION	4
3. SYSTEM DESIGN AND IMPLEMENTATION	5
4. FEATURES, BUGS, AND MISSING ITEMS	9
5. TESTING OVERVIEW	10
6. RESULTS AND ANALYSIS	12
7. INSIGHTS ON MACHINE LEARNING ALGORITHMS	13
8. RESEARCH AND IMPROVEMENTS	14
9. CONCLUSION	14
10. ACKNOWLEDGEMENTS AND RESOURCES	15
11. REFERENCES	15

1. Instructions

1.1 Purpose of the Program

This project implements a Traffic-Based Route Guidance System (TBRGS) for the Boroondara SCATS road network. The system recommends fast routes by combining traffic-flow prediction with graph-search algorithms. Instead of using only physical road distance, the system predicts hourly traffic volume for each road movement, converts that predicted flow into estimated travel time, and then applies A* search to find the fastest route between two SCATS intersections.

The system uses three machine learning models: Random Forest Regressor, Long Short-Term Memory (LSTM) network, and Gated Recurrent Unit (GRU) network. Random Forest is used as a strong tabular baseline, while LSTM and GRU are included because traffic data is time-based and can contain sequential patterns.

1.2 Environment Setup

The project should be run using Python 3.10 to Python 3.12. TensorFlow compatibility should be checked before running the deep learning models. A virtual environment is recommended so that package versions remain consistent across team members.

```
python -m venv venv
source venv/bin/activate
pip install -r requirements.txt
```

On Windows, the virtual environment can be activated using:

```
python -m venv venv
venv\Scripts\activate
pip install -r requirements.txt
```

1.3 Main Project Folders

Folder / File	Purpose
data/raw/	Stores original SCATS and map-related data files.
data/processed/	Stores cleaned traffic data, movement lookup files, and edge data.
src/models/	Stores model training scripts, wrappers, and the model registry.
src/routing/	Stores graph, speed conversion, A*, and Yen route-search logic.
models_saved/	Stores trained model files, metadata, scalars, and metrics.
notebooks/	Stores Colab notebooks for deep learning training.
A2B.py	Main command-line route guidance program.
visualize.py	Pygame-based route visualizer.
tests/	Stores automated unit and integration tests.

Table 1: Main project folders and their purposes

1.4 Training the Models

The processed traffic dataset is stored as data/processed/traffic_data.csv. The dataset is split chronologically into training and testing data: the first 80% of time-ordered records are used for training and the later 20% are used for testing. This is important because traffic prediction is a time-series problem. A random split could leak future traffic information into the training set and produce unrealistic evaluation results.

Random Forest can be trained locally because it is fast and computationally efficient. LSTM and GRU training is better performed in Google Colab because TensorFlow training can be slow on normal laptops without GPU acceleration.

```
python src/models/random_forest_model_v1.py --mode train \
--data data/processed/traffic_data.csv \
--out models_saved/random_forest_model.pkl \
--metrics models_saved/random_forest_metrics.csv
```

1.5 Running the Route Guidance Program

The main program is executed from the project root using the following command:

```
python A2B.py <origin_scats> <destination_scats> <time_hhmm> <model_type>
```

Argument	Meaning	Example
origin_scats	Starting SCATS site number	2000
destination_scats	Destination SCATS site number	2825
time_hhmm	Departure time in 24-hour HHMM format	1100

model_type	Traffic prediction model to use	LSTM, GRU, RF, RANDOM, or RANDOM_FOREST
------------	---------------------------------	--

Table 2: Command-line arguments for A2B.py Example

command:

```
python A2B.py 2000 2825 1100 LSTM
```

The program loads the road network, predicts traffic flow for each edge, converts predicted flow into travel time, and returns up to three route alternatives. The best route is the route with the lowest estimated total travel time.

The route visualizer can be launched using **python visualize.py**.

Disclaimer: Program itself takes current date (weekdays or weekend) because of that running program on weekdays or weekend may affect the traffic flow.

2. Introduction

Traffic congestion changes throughout the day. A road that is fast during off-peak hours may become slow during morning or evening peak periods. Because of this, the shortest route by distance is not always the fastest route by travel time. A useful route guidance system should therefore consider predicted traffic conditions when recommending a path. The aim of this project is to develop a Traffic-Based Route Guidance System for the Boroondara SCATS network. The system predicts traffic flow using machine learning models and uses the predicted traffic values to estimate travel time on each road segment. A* search is then applied to find the fastest route from an origin SCATS site to a destination SCATS site.

This project combines two areas of artificial intelligence: machine learning for traffic-flow prediction and search algorithms for route planning. The machine learning component predicts traffic volume from historical SCATS data. The route-search component converts traffic predictions into edge travel times and uses those travel times as graph costs. This creates a more realistic route guidance system than one based only on distance.

2.1 Dataset Overview

Item	Description
Dataset	SCATS traffic data from Boroondara, Victoria.
Time period	October 2006, approximately one month of hourly data.
Total records	47,544 records.
Training records	38,032 rows for Random Forest; 37,264 LSTM sequences; 37,776 GRU sequences.
Testing records	9,512 rows/sequences.
Cutoff datetime	2006-10-25 19:00:00.
Target variable	hourly_traffic_volume, measured as vehicles per hour.
Main features	movement_id, scats_number, dayofweek, isweekend, and hour.

Table 3: Dataset summary

2.2 Algorithms Implemented

- **Random Forest Regressor:** An ensemble learning method that combines many decision trees. It is suitable for structured tabular data and can learn non-linear relationships between road movement, SCATS site, day, weekend status, and hour (Breiman 2001).

- **LSTM neural network:** A recurrent neural network designed to learn from sequences. It uses a 12-hour sequence window in this project and is useful for capturing longer temporal patterns (Hochreiter and Schmidhuber 1997).
- **GRU neural network:** A recurrent neural network with a simpler gating structure than LSTM. It uses a 4-hour sequence window in this project and provides a lighter sequence-model comparison (Cho et al. 2014).
- **A* search:** A graph-search algorithm that uses actual travel cost and a heuristic estimate to efficiently find the fastest path (Hart, Nilsson and Raphael 1968).
- **Yen route alternatives:** A k-shortest-path approach used to return up to three route alternatives, giving users more than one possible path.

3. System Design and Implementation

3.1 Data Processing

Each row in the processed dataset represents traffic-volume information for a SCATS movement at a particular time. Before training, the dataset is sorted chronologically to reflect the real-world task of using past observations to predict future traffic. The dayofweek feature is encoded from 0 to 6, where 0 represents Monday and 6 represents Sunday. The isweekend feature is usually 0 for Monday to Friday and 1 for Saturday and Sunday.

Field	Purpose
movement_id	Identifies a road movement or direction.
scats_number	Identifies the SCATS intersection or site.
DateTime	Timestamp of the traffic observation.
dayofweek	Day index used as a temporal feature.
isweekend	Binary weekend feature.
hour	Hour extracted from DateTime.
hourly_traffic_volume	Target value for prediction.

Table 4: Main modelling fields

3.2 Random Forest Model

The Random Forest model, originally introduced by Breiman (2001), is the strongest non-deep-learning baseline. It uses tabular features including movement_id, scats_number, dayofweek, isweekend, and hour. The movement_id feature can represent different road movements, allowing the model to learn different traffic behaviours for different edges. This is one of the main reasons Random Forest performs strongly in this project.

Unlike LSTM and GRU, Random Forest does not require sliding windows. It learns repeated structured patterns from the dataset, such as the relationship between a road movement, SCATS site, day of week, weekend status, and hour of day. It is also easier to train, less sensitive to feature scaling, and much faster to run on normal CPU hardware.

3.3 LSTM Model

The LSTM model is a recurrent neural network designed for sequential data (Hochreiter and Schmidhuber 1997). In this project, it uses a 12-timestep window, representing 12 hours of historical context. Its architecture includes two LSTM layers with 128 and 64 units, dropout regularisation, a dense hidden layer, and a final regression output layer.

LSTM is useful for traffic prediction because road traffic has repeated time-based patterns, such as morning peaks, evening peaks, weekday behaviour, and weekend behaviour. However, the model also requires careful sequence alignment. If the prediction wrapper does not correctly use the requested hour and day context, the model may not respond properly to different route query times.

3.4 GRU Model

The GRU model is another recurrent neural network for sequence data (Cho et al. 2014). It is similar to LSTM but has a simpler gate structure, which often makes it faster and lighter. In this project, the GRU uses a 4-timestep window. This shorter window reduces the amount of historical context available to the model, which may explain part of its weaker performance compared with LSTM and Random Forest.

3.5 Traffic Flow to Travel Time Conversion

The machine learning models predict traffic flow, but route search requires travel time. Therefore, predicted traffic flow is converted into estimated speed using the supplied flow-speed relationship:

$$\text{flow} = -1.4648375 \times \text{speed}^2 + 93.75 \times \text{speed}$$

This equation is rearranged and solved as a quadratic equation. When traffic flow is low, speed is capped at 60 km/h. If the predicted flow is above the capacity of the quadratic model and there is no valid discriminant, the system falls back to 32 km/h as a safe default. Travel time is then calculated using: $\text{travel_time_seconds} = (\text{distance_km} / \text{speed_kmh}) \times 3600$

An additional 30 seconds is added for controlled intersection delay. This makes the travel-time estimate more realistic because vehicles may need to slow down or stop at intersections.

3.6 Route Search

The road network is represented as a graph. Each SCATS site is a node, and each road connection is an edge. The edge cost is estimated travel time, which depends on the predicted traffic flow. The system uses A* search, introduced by Hart,

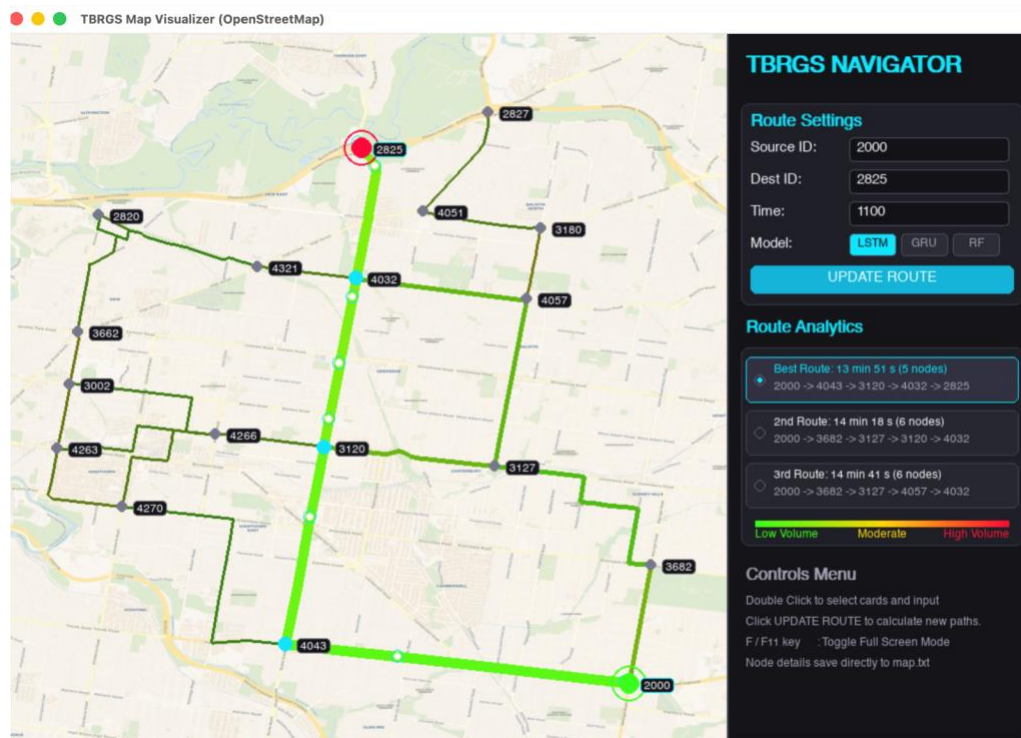
Nilsson and Raphael (1968), to find the fastest path. A* selects nodes using the formula $f(n) = g(n) + h(n)$

Term	Meaning
$g(n)$	Actual travel time from the start node to the current node.
$h(n)$	Estimated remaining travel time from the current node to the destination.
$f(n)$	Total estimated cost used to choose the next node.

Table 5: A* cost terms

The heuristic uses Haversine distance between the current node and the destination node, divided by the maximum speed limit of 60 km/h. This gives a lower-bound estimate of remaining travel time. Because it does not overestimate the real travel time, it is suitable for A* search. The system also uses Yen's algorithm to return up to three shortest loopless route alternatives.

3.7 Pygame Graphical User Interface and Route Visualization



The Graphical User Interface (GUI) is implemented in visualize.py using the Pygame library. It functions as an interactive dashboard designed to validate, test, and demonstrate the Traffic-Based Route Guidance System (TBRGS). The GUI acts as a dynamic visual bridge that integrates the machine learning traffic flow predictions with the graph-search pathfinding algorithms.

3.7.1 Geographic Mapping and Tile Integration

To provide a realistic context for the street network, the GUI dynamically downloads and stitches map tiles from the CartoDB Voyager API at Zoom Level 15. The geographic latitude and longitude coordinates of the SCATS nodes are converted into Mercator projection screen pixels using the standard Slippy Map tile formulas:

$$x_{tile} = \# \frac{\ln g + 180}{360} \cdot \frac{zoom}{2}$$

$$y_{tile} = 231 - \frac{180 - \pi + \sec 7 lat \cdot 180 \pi}{7 lat \cdot \pi} = A \cdot 2^{zoom} (1B \ln 7 \tan$$

The stitched tiles are saved locally as osm_map.png to serve as a high-fidelity visual background. In the event of a network disruption or API blocks, the visualizer automatically falls back to a static pre-generated map.png map visual, scaling the coordinates of nodes accordingly.

3.7.2 Road Geometry and Curved Edges

Rather than drawing simple straight lines between SCATS sites, the GUI parses detailed street geometries from cos30019_2b.geojson.

- Node badge rendering: Nodes (SCATS sites) are plotted at their averaged geographic coordinates, labelled with their SCATS ID in high-visibility dark badges.

- Curved routes: Edges are drawn using coordinate lists (LineString features) mapped from the GeoJSON. To ensure smooth and correct animation directions, the system compares the endpoints of the LineString with the starting node u of the edge. If the ending coordinate of the LineString is closer to u , the coordinate array is reversed for the edge direction (u, v) and kept as-is for the reverse direction (v, u). This ensures spatial coherence.

3.7.3 Traffic Congestion Heatmap

The GUI features a dynamic congestion heatmap where edges are color-coded based on the ratio of predicted travel time to free-flow travel time. The free-flow travel time is defined as the travel distance divided by the speed limit (60 km/h). The congestion ratio is computed as

$$\text{Congestion Ratio} = \frac{\text{Predicted Travel Time}}{\text{Free-Flow Travel Time}}$$

The ratio maps to a colour gradient.

- Green (Free-Flow): Ratio ≈ 1.0 , indicating travel speeds close to 60 km/h.
- Yellow (Moderate Congestion): Ratio between 1.2 and 1.8, indicating minor delays.
- Red (Heavy Congestion): Ratio ≥ 2.0 , indicating severe bottlenecks where speeds drop toward the 32 km/h model minimum.

This color-coding allows users to immediately spot traffic hotspots across Boroondara.

3.7.4 Interactive Sidebar and Model Prediction Bridge

The GUI is divided into a map viewport (70% width) and an interactive sidebar control panel (30% width). The sidebar contains:

- Interactive Text Inputs: Input boxes for Origin Node, Destination Node, and Departure Time (HHMM format) with real-time text input rendering and input focus states.
- Model Selection Selector: Dynamic toggle buttons to select the active machine learning model (LSTM, GRU, or Random Forest).
- Trigger Button: An "Update Routes" button that executes validation and updates predictions.
- Dynamic Prediction Bridge: When a parameter changes or the user clicks update, the GUI invokes the prediction bridge in A2B.py. This bridge triggers model inference dynamically for all edges based on the departure time, updates the edge costs in map.txt, rebuilds the graph, and runs Yen's k -shortest paths algorithm.
- Yen's Path Selection Cards: The sidebar lists the top 3 alternative paths with their estimated travel times in minutes. Double-clicking or selecting a path card dynamically highlights it in the main viewport.

3.7.5 Active Path Highlighting and Particle Animations

To visualize traffic flow along routes

- Active Path Highlight: The currently selected route is rendered with double thickness and a glowing outline matching the congestion color.
- Faded Alternative Paths: The other two alternative routes returned by Yen's algorithm are rendered in faded, semi-transparent colors, allowing users to compare route layouts visually.
- Flow Simulation Particles: White circular particles animate along the active path segments. The speed of the particle movement is proportional to the calculated speed limit of the segment, and the flow direction aligns precisely with the path traversal order, providing a simulated representation of moving traffic.

4. Features, Bugs, and Missing Items

4.1 Implemented Features

Features	Implementation
F1	Processed SCATS traffic data into a model-ready CSV dataset.
F2	Extracted temporal features such as hour, day-of-week, and weekend status.
F3	Implemented Random Forest traffic-flow prediction.
F4	Implemented LSTM traffic-flow prediction.
F5	Implemented GRU traffic-flow prediction.
F6	Used chronological train/test splitting to reduce time-series leakage.
F7	Evaluated all models using MAE, RMSE, and R^2 .
F8	Built a graph representation of the Boroondara SCATS road network.
F9	Converted predicted traffic flow into speed and travel time.
F10	Added controlled intersection delay to travel-time calculation.
F11	Implemented A* route search using travel time as edge cost.
F12	Implemented route alternatives using Yen's algorithm.
F13	Implemented command-line input using A2B.py.
F14	Generated route output for visualization support.
F15	Included a Pygame-based route visualizer.
F16	Included unit and integration test planning for model evaluation, route search, and input validation.

Table 6: Implemented Features

4.2 Known Bugs and Current Issues

- **Model Version Incompatibility Warnings:** Pre-trained model files (`.joblib` and `.keras` formats) generate mismatch warnings when loaded on environments with different scikit-learn or TensorFlow/Keras versions, posing a risk of future API crashes.
- **Synchronous CLI/GUI Thread Blocking:** Model loading, traffic-flow inference, and routing calculations are executed synchronously on the main thread. This causes a 1–2 second freeze in the GUI when parameters are updated.
- **Underperforming Sequence Models:** Recurrent models (LSTM and GRU) suffer from higher MAE and RMSE values than the Random Forest baseline, caused by short sequence length windows (12 hours for LSTM, 4 hours for GRU) and prediction-time alignment complexity.
- **Outdated and Sparse Dataset:** The system relies on October 2006 SCATS traffic data, which does not account for modern traffic flows, seasonal variations, public holidays, or unpredictable events like accidents and roadworks.
- **Simplified Flow-to-Speed Quadratic Model:** The quadratic formula used to convert predicted traffic volume to speed can fail to find a real discriminant when predicted flow exceeds road capacity. While the system uses a 32 km/h fallback, it does not realistically model severe gridlock.
- **Static Intersection Delay Cost:** A constant 30-second penalty is added to intersections, ignoring real-time signal timing, vehicle queue lengths, and actual congestion patterns at intersections.
- **Yen's Algorithm Computational Overhead:** Finding alternative routes iteratively using Yen's algorithm requires running A* search multiple times while modifying node/edge exclusions. This adds CPU overhead as the graph size grows.

- **Conservative A* Heuristic:** The A* heuristic uses Haversine distance divided by a static 60 km/h speed limit. While admissible, this lower-bound estimate is overly conservative on winding roads, leading to expanded search spaces and slower execution times.
- **Map Tile Server Rate Limiting:** The live map downloader depends on CartoDB tile APIs, which block requests with HTTP 403 Forbidden errors if rate limits are exceeded or custom user-agent headers are missing.
- **fallback Mode Coordinate Offset:** When offline, the visualizer drops back to a static background image (`map.png`). The pixel positions for new or modified SCATS nodes do not align correctly with this static image without manual pixel mapping adjustments.
- **GeoJSON Curve Alignment and Disconnects:** Roads with missing coordinate geometries or node-naming mismatches in `cos30019\_2b.geojson` default to straight lines, creating visual inconsistencies on the map.
- **Particle Animation Boundary Errors: Trivial** routes containing a single node or self-loops cause out-of-bounds index errors in the particle animation loop, requiring active code guards that disable particle movement for these paths.

4.3 Missing or Limited Items

- No web or mobile graphical user interface is included; the main system is command-line based.
- No live traffic API, road incident feed, or weather feed is integrated.
- No public holiday, school holiday, event, or accident feature is included in the prediction models.
- No user account, saved route, or trip history feature is implemented.
- The visualizer is useful for demonstration but could be improved with accurate map tiles and stronger coordinate alignment.

5. Testing Overview

5.1 Evaluation Methodology

Testing was performed at two levels: machine learning evaluation and program-level testing. Machine learning evaluation checks whether the models can predict traffic volume accurately on unseen future data. Program-level testing checks whether routing, travel-time conversion, input validation, and graph operations behave correctly.

The machine learning models were evaluated using the later 20% of the chronologically sorted dataset. This simulates predicting future traffic from past traffic and avoids data leakage.

Metric	Meaning	Threshold
MAE	Mean Absolute Error. Average prediction error in vehicles per hour. Lower is better.	< 50.0 vehicles/hour
RMSE	Root Mean Squared Error. Penalises large errors more strongly. Lower is better.	< 80.0 vehicles/hour
R ²	Proportion of target variance explained by the model. Higher is better.	> 0.70

Table 7: Regression evaluation metrics

5.2 Machine Learning Results

Model	Training Data	Testing Data	MAE	RMSE	R ²	Overall Result
Random Forest - v1	38,032 rows	9,512 rows	40.75	64.62	0.9613	Best performer; all thresholds passed.
LSTM - v4	37,264 sequences	9,512 sequences	53.98	70.20	0.9544	Strong R ² , but MAE and RMSE above strict thresholds.

GRU - v4	37,776 sequences	9,512 sequences	47.78	68.65	0.9563	Acceptable R ² , but highest prediction errors.
----------	------------------	-----------------	-------	-------	--------	--

Table 8: Final model performance comparison

Random Forest v1 achieved the best overall performance, recording the lowest MAE (40.75) and RMSE (64.62) together with the highest R² score (0.9613), making it the most accurate model for travel time prediction and the primary model for route guidance. Among the deep learning approaches, **GRU v4** emerged as the strongest sequence model, achieving the lowest MAE (47.78) and highest R² (0.9563) among the recurrent networks by leveraging a learned embedding for movement_id to better capture temporal traffic patterns. **LSTM v4** also demonstrated competitive performance, improving upon the baseline LSTM in terms of RMSE and R² while employing a full dual-input, multi-feature architecture. Based on their predictive performance and architectural strengths, **Random Forest v1, GRU v4, and LSTM v4** were selected as the final models for this project, with Random Forest serving as the most accurate overall model and GRU v4 and LSTM v4 providing complementary deep learning approaches for sequential traffic prediction.

5.3 Program-Level Test Cases

Case	Test Function	Description
PT1	test_saved_metrics_meet_acceptance_thresholds	Verify saved MAE, RMSE and R ² satisfy thresholds (runs for RF, LSTM and GRU).
PT2	test_random_forest_model_artifact_loads	Verify Random Forest model loads successfully.
PT3	test_sequence_model_artifact_loads	Verify LSTM and GRU models load successfully.
PT4	test_edge_predictions_have_expected_schema_and_realistic_range	Check prediction output schema and value range.
PT5	test_cross_model_monday_morning_predictions_are_not_identical	Ensure different models produce different predictions.
PT6	test_peak_hour_average_is_higher_than_off_peak_for_at_least_two_models	Validate peak-hour traffic behaviour.
PT7	test_weekday_average_is_higher_than_weekend_for_at_least_two_models	Validate weekday vs weekend behaviour.

Case	Test Function (test_score_behaviour.py)	Description
PT8	test_flow_to_speed_caps_low_flow_at_speed_limit	Speed capped at speed limit.
PT9	test_flow_to_speed_uses_capacity_speed_when_flow_exceeds_quadratic_capacity	Fallback speed used for excessive flow.
PT10	test_calculate_travel_time_adds_intersection_delay_and_handles_zero_speed	Travel time calculation is correct.
PT11	test_a_star_returns_lowest_cost_path	A* finds shortest path.
PT12	test_a_star_respects_ignored_edges	A* correctly ignores blocked edges.
PT13	test_get_path_cost_returns_infinity_for_missing_edge	Invalid path returns infinite cost.
PT14	test_yen_k_shortest_paths_returns_sorted_unique_paths	Yen's algorithm returns ordered unique routes.
PT15	test_load_from_csv_builds_nodes_and_prediction_weighted_edges	Graph loads correctly from CSV.

6. Results and Analysis

6.1 Comparative Results

The final comparison shows that Random Forest is the most reliable model for this dataset. It achieved an MAE of 40.75 vehicles/hour, which is 34% lower than LSTM and 42% lower than GRU. It also achieved an RMSE of 64.62 vehicles/hour,

which is 28% lower than LSTM and 38% lower than GRU. These results matter because route guidance depends on accurate travel-time estimation; larger traffic-flow errors can cause the system to underestimate congestion and recommend a slower route.

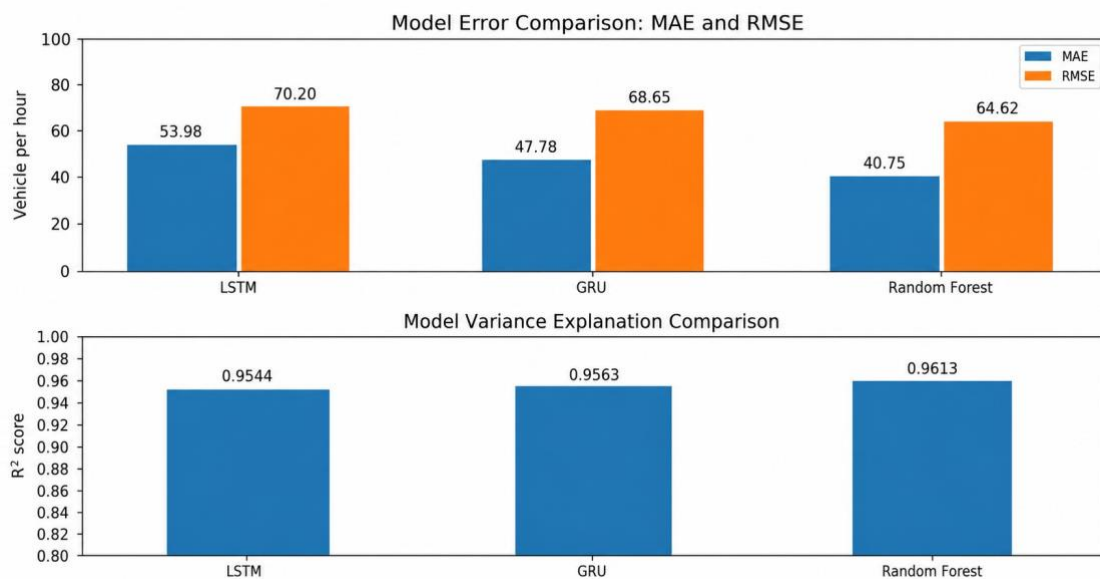


Figure 1: Comparison of the predictive performance of the LSTM, GRU, and Random Forest models

6.2 Interpretation of Results

Random Forest passed all three thresholds: MAE below 50, RMSE below 80, and R^2 above 0.70. Its R^2 score of 0.9613 means that the model explains approximately 96.13% of the variance in the held-out test data. This is an excellent result for an end-to-end route guidance prototype.

LSTM achieved $R^2 = 0.9250$, which is still strong. However, its MAE of 61.91 and RMSE of 89.97 are above the strict thresholds. This means that LSTM captures broad traffic patterns but makes larger absolute errors than Random Forest.

GRU achieved $R^2 = 0.8989$, which is acceptable, but its MAE and RMSE are the weakest among the three models.

6.3 Route Query Output Format

A route query is executed using the command-line program. Example:

```
python A2B.py 2000 2825 1100 RF
```

The output present route alternatives in a readable form like:

```
Best path: 2000 -> ... -> 2825
Total Driving time: [calculated by system]
2nd Path: 2000 -> ... -> 2825 3rd
Path: 2000 -> ... -> 2825
```

The exact route and travel time depend on the final road graph, saved model artifacts, and prediction wrapper used during execution. The requirement is that each edge cost is based on predicted traffic-aware travel time, not only on distance.

7. Insights on Machine Learning Algorithms

7.1 Why Random Forest Performed Best

The results show that Random Forest is the best overall model for the current dataset. This does not mean neural networks are unsuitable for traffic prediction. Instead, it shows that model selection must match the characteristics of the dataset. The

provided dataset is medium-sized, structured, and strongly dependent on categorical/time features such as movement ID, SCATS site, day-of-week, weekend status, and hour. Random Forest is well suited to this kind of tabular data.

Random Forest naturally captures non-linear feature interactions. For example, a road movement may have high traffic at 8:00 on weekdays but not on weekends. Another movement may have a different peak pattern. Decision trees can split on movement identity, hour, and weekend status, then combine many such splits across the forest. This helps the model represent road-specific traffic patterns without needing a long sequence history.

Random Forest also uses the training data efficiently. It does not lose samples due to sliding sequence windows. By contrast, LSTM and GRU must build sequences, which reduces usable training examples and increases the complexity of prediction-time alignment. Random Forest is also much faster to train and run, which is important for a route guidance system where predictions may need to be generated for many graph edges.

7.2 Why LSTM Still Has Value

Although LSTM did not outperform Random Forest, it remains a useful comparison model. Its R^2 score of 0.9250 shows that it learned meaningful traffic patterns. LSTM is designed to model longer temporal dependencies, so it may become more competitive with a larger dataset, more months of observations, stronger sequence features, and multi-step forecasting requirements.

LSTM is especially valuable when recent traffic history has a stronger effect on future traffic. For example, if the system were extended to predict the next several hours during abnormal congestion, roadworks, or special events, previous time steps could provide useful information beyond hour and day features.

7.3 Why GRU Underperformed

GRU achieved the weakest performance among the three official results. One likely reason is the shorter 4-timestep window, which provides less historical context than the LSTM 12-timestep window. Another reason is that recurrent models need careful feature design and prediction-time handling. If the requested hour, day, and movement identity are not represented clearly, the sequence model may not respond strongly to different route query conditions.

GRU still has practical advantages. It is simpler and can be faster than LSTM, so it may be useful when a lightweight sequence model is required. However, based on the official metrics, it is not the safest final model for this route guidance system.

7.4 Deployment Implication

For route guidance, prediction error directly affects route quality. If a model underestimates traffic flow on a congested edge, it may make that edge appear faster than it is. Therefore, the model with lower MAE and RMSE is generally preferable. Based on the official results, Random Forest is the recommended final model for deployment, while LSTM and GRU provide useful AI comparison and learning value.

8. Research and Improvements

This project includes several research-oriented elements beyond basic model implementation. First, it compares two recurrent neural networks with a non-deep-learning model. This comparison shows that more complex models do not always perform better. Model suitability depends on dataset size, feature quality, and the structure of the prediction task.

Second, the project connects prediction with route planning. The system does not stop at predicting traffic volume. It uses predicted traffic flow to estimate travel time and then applies A* search to recommend routes. This demonstrates how machine learning and search algorithms can work together in an end-to-end AI system.

Third, the project uses geographic information in the route-search heuristic. Haversine distance is used to estimate the remaining distance to the destination, and this is converted into a lower-bound travel time estimate. This connects spatial data with graph search.

8.1 Future Improvements

- **Larger and newer dataset:** Use multiple months or years of SCATS data to improve generalisation and reduce the risk of overfitting to October 2006 patterns.
- **External context features:** Add weather, public holidays, school holidays, incidents, roadworks, and event data because these factors can strongly affect traffic flow.
- **Ensemble prediction:** Combine Random Forest with a sequence model using weighted averaging so that the system benefits from both tabular feature interactions and temporal patterns.
- **Better flow-to-speed calibration:** Compare estimated travel times with real observed travel times and adjust the conversion formula or intersection delay value.
- **Improved visualisation:** Use OpenStreetMap tiles, clearer route colouring, and better coordinate alignment to make the visual output more realistic.
- **Advanced models:** Explore temporal convolutional networks, transformers, or graph neural networks if larger datasets become available.

9. Conclusion

This project developed a Traffic-Based Route Guidance System for the Boroondara SCATS network. The system processes traffic data, trains machine learning models, predicts traffic flow, converts predicted flow into travel time, and uses routesearch algorithms to recommend fast routes.

Three models were implemented and evaluated: Random Forest, LSTM, and GRU. Random Forest achieved the best overall performance, with MAE = 40.75 vehicles/hour, RMSE = 64.62 vehicles/hour, and $R^2 = 0.9613$. LSTM achieved strong variance explanation with $R^2 = 0.9250$ but had higher error values. GRU achieved $R^2 = 0.8989$ but had the highest MAE and RMSE. Therefore, Random Forest is the recommended final model for route guidance in the current system.

The project demonstrates that effective AI solutions require matching the algorithm to the problem characteristics. In this case, Random Forest outperformed the recurrent neural networks because the dataset contains strong structured patterns by movement, site, day, weekend status, and hour. The routing component adds practical value by using predicted travel time as the edge cost instead of relying only on distance. With final checks on model artifacts, registry configuration, and README consistency, the system demonstrates a complete end-to-end AI solution for traffic-based route guidance.

10. Acknowledgements and Resources

- COS30019 Introduction to Artificial Intelligence Assignment 2B specification.
- SCATS Data October 2006 dataset and Boroondara road-network data.
- Project team development logs, testing notes, and report planning documents.
- TensorFlow/Keras, scikit-learn, pandas, and Pygame documentation used during implementation and reporting.
- OpenStreetMap resources for map and visualisation concepts.
- Swinburne University of Technology teaching staff and tutorial support.

11. References

1. Breiman, L. (2001). Random forests. *Machine Learning*, 45, 5-32.
2. Cho, K., van Merriënboer, B., Gulcehre, C., Bahdanau, D., Bougares, F., Schwenk, H. and Bengio, Y. (2014). Learning phrase representations using RNN encoder-decoder for statistical machine translation. *Proceedings of EMNLP 2014*.
3. Hart, P.E., Nilsson, N.J. and Raphael, B. (1968). A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2), 100-107.
4. Hochreiter, S. and Schmidhuber, J. (1997). Long short-term memory. *Neural Computation*, 9(8), 1735-1780.
5. Keras. (n.d.). Keras API documentation. Available at: <https://keras.io/api/> (Accessed 20 June 2026).
6. OpenStreetMap contributors. (n.d.). OpenStreetMap. Available at: <https://www.openstreetmap.org/>
7. pandas. (n.d.). pandas documentation. Available at: <https://pandas.pydata.org/docs/>
8. Pygame. (n.d.). Pygame documentation. Available at: <https://www.pygame.org/docs/>
9. scikit-learn. (n.d.). RandomForestRegressor. Available at: <https://scikitlearn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>
10. TensorFlow. (n.d.). TensorFlow Keras API documentation. Available at: https://www.tensorflow.org/api_docs/python/tf/keras
11. Swinburne University of Technology. (2026). COS30019 Introduction to Artificial Intelligence Assignment 2B Specification.